

Experiment # 03

Implementation of Profiling in C++ using Timer

Objective:

The objective of this lab is to profile the complex code using timer header file to get more accurate execution time.

Software Tools:

- Ubuntu

Theory:

Timer:

Although the GPROF method provides more detailed information than the first few methods presented, it is often necessary to measure execution time with finer granularity than a function. Suppose gprof was used and it shows that 90% of the time is spent in one subroutine. That subroutine becomes the primary target for optimization. But if the routine includes several loops, the next step is then to identify the most time-consuming parts within that subroutine. A possible approach is to use the `timer()` function, as provided by many operating systems, including UNIX. In this case, however, the program must be instrumented such that the timer is read at the beginning and end of the code segment being measured. Instrumenting the code means adding lines of code explicitly to perform the timing measurements. Such lines of code are temporary and are removed once the desired data has been collected. This method is useful for fine-grain measurements, such as a code segment or loop, but it is not as convenient as gprof to obtain measurements of multiple functions or processes at once. Here is an example of a program that uses `timer()`.

Procedure:

Enter the Remote host IP address and your Host username. Click OK

OR

Log-in to your personal machine.

Insert your Password and you are logged in.



After successful login, clear the screen by typing “clear” in the command window. Verify that, you are at your home directory.

```
fasih@fasih-VirtualBox:~$
```

Now create a new directory with the “mkdir” for this lab.

It will create the new directory named “lab03”

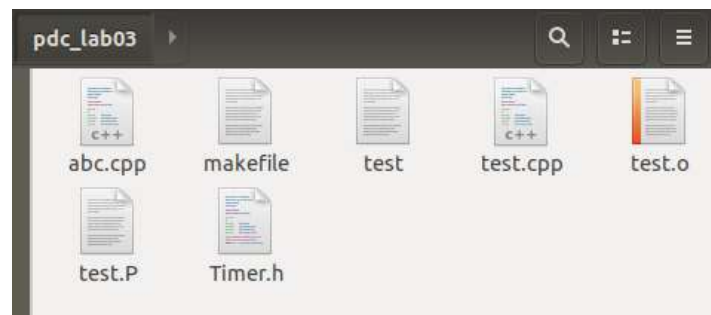
Now change your directory to lab03 by using “cd” command

```
fasih@fasih-VirtualBox:~$ mkdir pdc_lab03
fasih@fasih-VirtualBox:~$ cd pdc_lab03
fasih@fasih-VirtualBox:~/pdc_lab03$
```

Here you have to Paste the example code given to you. Simply drag from your windows directory to the MobaXterm directory Panel.

OR

Download the files in your local linux environment & place in this directory.



Example: -



```
test.cpp
~/pdc_lab03

#include <iostream>
#include <vector>
#include "Timer.h"
#define N 1e7
using namespace std;
int main()
{
    Timer aTimer("Array Time"), vTimer("Vector Time");
    cout << "C++ source file" << endl;
    int * Array = (int *) malloc(N * sizeof(int) );
    vector<int> Vector(N);
    aTimer.Start();
    for(int i=0; i<N; ++i)
    {
        Array[i] = i;
    }
    aTimer.Stop();
    free(Array);
    vTimer.Start();
    for(int i=0; i<N; ++i)
    {
        Vector[i] = i;
    }
    vTimer.Stop();
    aTimer.Print();
    vTimer.Print();
    return 0;
}
```

C++ ▼ Tab Width: 8 ▼

In make file change the source file name to execute the example code. Now, All you have to do is write a command “make all”.

```

fasih@fasih-VirtualBox:~/pdc_lab03$ make all
rm -f test test.o *~ *.P
g++ -O3 -MD -I. -c test.cpp -o test.o
g++ test.o -o test -L. -lm
./test
C++ source file
Array Time : 91.791 msec
Vector Time : 71.53 msec
fasih@fasih-VirtualBox:~/pdc_lab03$

```

It will show you the execution time taking by required chunk of code we are interested in. So, we can see that by including Timer library we determined the exact time that one function or chunk of code took.

Example: -

```

#include<iostream>
#include<Timer.h>
using namespace std;
long long int func_A (long int);
void func_B (long long int);
int main (void)
{
    Timer aTime("Function A: ");
    Timer bTime("Function B: ");
    long long int num;
    long int repeats=4000000;
    aTime.Start();
    num = func_A(repeats);
    printf("X (func_A) = %lld \n",num);
    aTime.Stop();
    bTime.Start();
    func_B(num);
    bTime.Stop();
    aTime.Print();
    bTime.Print();
    return 0;
}
long long int func_A (long int times)
{
    long long int x = 0;
    while(x<times)
    {
        x++;
    }
}

```

```

}
times=times*2;
while(x<times)
{
x++;
}
times=times*2;
return x;
}
void func_B (long long int times)
{
long long int x = 0;
while(x<times)
{
x=x+2;
}
printf("X (func_B) = %lld\n",x);
}

```

In make file change the source file name to execute the example code. Now, All you have to do is write a command “make all”.

```

fasih@fasih-VirtualBox:~$ cd test_example
fasih@fasih-VirtualBox:~/test_example$ gedit test.cpp
fasih@fasih-VirtualBox:~/test_example$ make all
rm -f test test.o *~ *.P
g++ -O3 -MD -I. -c test.cpp -o test.o
g++ test.o -o test -L. -lm
./test
X (func_A) = 8000000
X (func_B) = 8000000
Function A: : 0.207 msec
Function B: : 0.106 msec
fasih@fasih-VirtualBox:~/test_example$

```

Lab task:

1. Run the provided test example code.
 2. Write a C++ program to add, subtract, multiply and divide two random number in separate functions. Take a size of 1 Million for each function.
- Find the time taking by each function using Timer library.